Algorithmes et Pensée Computationnelle

Programmation orientée objet - Exercices avancés

Le but de cette séance est de découvrir et de mettre en pratique un paradigme de programmation largement utilisé : la programmation orientée objet (POO). Ce paradigme repose sur la création d'objets qui regroupent des données et des comportements, et qui peuvent interagir entre eux. Au fil des exercices, vous apprendrez à manipuler des objets et à maîtriser les notions fondamentales de classe, d'attribut, de méthode et d'encapsulation. À l'issue de cette séance, vous serez capables de concevoir des programmes mieux structurés et plus faciles à maintenir.

Les exercices de cette séance seront réalisés en Java et en Python.

Les durées indiquées (🕓) sont données à titre indicatif.

1 Interaction entre plusieurs instances d'une même classe - Java (30 minutes)

Dans cette section, nous allons simuler un jeu de combat entre deux protagonistes représentant des instances d'une classe `Fighter` que nous allons créer. Chaque `Fighter` aura des attributs qui le définissent. Ces attributs sont :

- `name` : *(String)* chaque combattant sera identifié par un nom unique.
- `health` : *(int)* représentant le nombre de points de vie d'un combattant. Il contient des valeurs comprises entre 0 et 10. À l'instanciation de l'objet, le combattant a 10 points de vie par défaut.
- `attack` : *(int)* représentant une valeur qui sera utilisée pour calculer le nombre de points de dégâts infligés à l'adversaire.
- `defense` : *(int)* représentant une valeur qui sera utilisée pour calculer le nombre de points de dégâts reçus.

Deux attributs de classe seront également utilisés :

- `instances` : Liste comprenant les combattants qui ont été instanciés et qui sont toujours en vie.
- `attackModifier` : Dictionnaire comportant 3 types d'attaques, chacune modifiant les dégâts qui vont être infligés. Les trois types d'attaques sont `poing, pied et tête` modifiant respectivement l'attaque par 1, 2, 3.

Le but de cette partie est d'étudier les interactions entre deux instances d'une même classe. Cette classe se présentera sous la forme d'un `Fighter`. Chaque instance de la classe `Fighter` pourra attaquer les autres instances.
Vous devrez compléter les 4 méthodes suivantes :

1. `isAlive()`
2. `checkDead()`
3. `checkHealth()`
4. `attack(String type, Fighter other)`

Voici le squelette du code (à télécharger sur Moodle) :

```
1  import java.util.HashMap;
2  import java.util.List;
3  import java.util.ArrayList;
4  import java.util.Map;
5
6  public class Fighter {
7      private String name;
8      private int health;
```

```
9       private int attack;
10      private int defense;
11      private static List<Fighter> instances = new ArrayList<Fighter>();
12      private static HashMap<String, Integer> attackModifier = new
        HashMap<String, Integer>(
13              Map.of("poing", 1, "pied", 2, "tete", 3));
14
15      public Fighter(String name, int health, int attack, int defense) {
16          this.name = name;
17          this.setHealth(health);
18          this.attack = attack;
19          this.defense = defense;
20          instances.add(this);
21      }
22
23      public int getAttack() {
24          return attack;
25      }
26
27      public int getHealth() {
28          return health;
29      }
30
31      public int getDefense() {
32          return defense;
33      }
34
35      public String getName() {
36          return name;
37      }
38
39      public void setAttack(int attack) {
40          this.attack = attack;
41      }
42
43      public void setDefense(int defense) {
44          this.defense = defense;
45      }
46
47      public void setHealth(int health) {
48          if (health < 0) {
49              this.health = 0;
50          } else if (health > 10) {
51              this.health = 10;
52          } else {
53              this.health = health;
54          }
55      }
56
57      public void setName(String name) {
58          this.name = name;
59      }
60
61      public boolean isAlive() {
62          // à compléter
63      }
64
65      public static void checkDead() {
```

```
66          // à compléter
67      }
68
69      public static void checkHealth() {
70          // à compléter
71      }
72
73      public void attack(String type, Fighter other) {
74          // à compléter
75      }
76  }
```

Question 1: (*5 minutes*) isAlive()
Définir une méthode `isAlive()` de type `boolean` qui retournera `true` si l'instance a plus que 0 points de vie et `false` si l'instance en a moins.

Solution

```
1  public boolean isAlive() {
2      if (this.health > 0) {
3          return true;
4      } else {
5          return false;
6      }
7  }
```

Question 2: (*10 minutes*) checkDead()
Définir une méthode `checkDead()` qui parcourt la liste des instances, et contrôle que chacune d'entre elle est encore en vie. Si ce n'est pas le cas, l'instance en question est supprimée de la liste des instances et le message "`nameInstance` est mort" sera affiché.

Conseil

Évitez de supprimer des éléments de instances pendant que vous la parcourez avec une boucle for each, car cela peut provoquer une `ConcurrentModificationException`. Utilisez une deuxième liste pour ne conserver que les combattants encore en vie.

Solution

```
1  public static void checkDead() {
2      // Initialisation de la liste de Fighter en vie
3          List<Fighter> temp = new ArrayList<Fighter>();
4          //Ici, on parcourt les instances de Fighter
5          for (Fighter f : Fighter.instances) {
6              // Et on fait appel à la méthode isAlive() pour
       vérifier que le Fighter est en vie
7              if (f.isAlive()) {
8                  temp.add(f);
9              } else {
10                 System.out.println(f.getName() + " est mort");
11             }
12         }
13         Fighter.instances = temp;
14     }
```

Question 3: (*5 minutes*) checkHealth()
Définir une méthode `checkHealth()` qui parcourt la liste des instances et affiche le nombre de points de vie qui reste au combattant sous le format "`nameInstance` a encore `healthInstance` points de vie".

Solution

```
1  public static void checkHealth() {
2          for (Fighter f : Fighter.instances) {
3              System.out.println(f.getName() + " a encore " +
       f.getHealth() + " points de vie");
4          }
5      }
```

Question 4: (*10 minutes*) attack(String type Fighter other)
Définir une méthode `attack(String type, Fighter other)` qui permettra de retirer des points de vie au combattant `other` en fonction de l'attaque de l'instance appelée, du type d'attaque sélectionné et de la défense de `other`.

Commencez par contrôler si `other` est encore en vie. Si tel n'est pas le cas, indiquez qu'il est déjà mort : "`otherName` est déjà mort".
Si `other` est encore en vie, retirez des points de vie à `other`. Le nombre de points de vie devant être retiré se calcule en utilisant la formule suivante : `attackModifier.get(type) * attackInstance - defenseOther`. Si le résultat du calcul est négatif, les dégâts sont ramenés à zéro : une attaque ne doit jamais augmenter les points de vie de l'adversaire. Appelez ensuite les fonctions `checkDead()` et `checkHealth()` afin d'avoir un aperçu des combattants restants et de leur santé.

Solution

```
1  public void attack(String type, Fighter other) {
2      if (other.isAlive()) {
3          int damage = Fighter.attackModifier.get(type) *
       this.attack - other.getDefense();
4          if (damage < 0) {
5              damage = 0;
6          }
7          other.setHealth(other.getHealth() - damage);
8          Fighter.checkDead();
9          Fighter.checkHealth();
10     } else {
11         System.out.println(other.getName() + " est déjà mort");
12     }
13 }
```

Pour terminer, vous pouvez exécuter le code ci-dessous (disponible dans le dossier Code sur Moodle) pour vérifier que votre programme fonctionne correctement :

```
1  public class Question4 {
2      public static void main(String[] args) {
3          Fighter p1 = new Fighter("P1", 10, 2, 2);
4          Fighter p2 = new Fighter("P2", 10, 2, 2);
5          Fighter p3 = new Fighter("P3", 10, 2, 2);
6          p1.attack("pied", p2);
7          p1.attack("poing", p2);
```

```
        p1.attack("tête", p2);
        p1.attack("tête", p2);
    }
}
```

Vous devriez obtenir ce résultat :

```
P1 a encore 10 points de vie
P2 a encore 8 points de vie
P3 a encore 10 points de vie
P1 a encore 10 points de vie
P2 a encore 8 points de vie
P3 a encore 10 points de vie
P1 a encore 10 points de vie
P2 a encore 4 points de vie
P3 a encore 10 points de vie
P2 est mort
P1 a encore 10 points de vie
P3 a encore 10 points de vie

Process finished with exit code 0
```

2 Notions d'héritage - Python

Question 5: (*30 minutes*) **Un exemple appliqué**
Dans cet exercice, nous allons modéliser le calcul de la rémunération de deux catégories d'enseignants universitaires : les professeurs à temps plein, représentés par la classe `Professeur`, et les intervenants à temps partiel, représentés par la classe `Collaborateur`. Les deux classes héritent de la classe mère `Enseignant`.

La classe `Enseignant` définit une méthode qui renvoie un salaire annuel moyen de 60 000 dollars américains pour un enseignant ayant moins de 10 ans d'expérience, et de 100 000 dollars à partir de 10 ans d'expérience. Pour un enseignant à temps partiel, cette méthode renvoie plutôt la chaîne de caractères `Le salaire annuel ne s'applique pas aux collaborateurs`.

Chaque classe fille définit également une méthode de calcul de la rémunération mensuelle :

- Pour un `Professeur`, elle correspond au salaire mensuel de base, auquel s'ajoute une commission pour chaque comité auquel il participe.
- Pour un `Collaborateur`, elle correspond au taux horaire multiplié par le nombre d'heures travaillées dans le mois.

Complétez le code ci-dessous :

```
class Enseignant:
    def __init__(self, name, years_experience, full_time):
        ...
    def salaire_annuel_moyen(self):
        ...

class Professeur(Enseignant):
    def __init__(self, name, years_experience, monthly_salary,
    commission, num_committees):
        ...
    def paye_mensuelle(self):
        ...

class Collaborateur(Enseignant):
    def __init__(self, name, years_experience, hours_per_month, rate):
        ...
```

```
16      def paye_mensuelle(self):
17          ...
18
19  prof1 = Professeur("Alexandra", 8, 3000, 200, 4)
20  prof2 = Collaborateur("David", 10, 40, 30)
21  # exemples
22  print(prof1.salaire_annuel_moyen())
23  print(prof2.paye_mensuelle())
```

Conseil

Pensez à redéfinir les attributs de la classe mère en utilisant `super().__init__()`.

Solution

```
class Enseignant:
    def __init__(self, name, years_experience, full_time):
        self.name = name
        self.years_experience = years_experience
        self.full_time = full_time

    def salaire_annuel_moyen(self):
        if self.full_time:
            if self.years_experience < 10:
                return 60000

            else:
                return 100000

        else:
            return "Le salaire annuel ne s'applique pas aux collaborateurs"


class Professeur(Enseignant):
    def __init__(self, name, years_experience, monthly_salary, commission, num_committees):
        super().__init__(name, years_experience, True)
        self.monthly_salary = monthly_salary
        self.commission = commission
        self.num_committees = num_committees

    def paye_mensuelle(self):
        return self.monthly_salary + self.commission*self.num_committees

class Collaborateur(Enseignant):
    def __init__(self, name, years_experience, hours_per_month, rate):
        super().__init__(name, years_experience, False)
        self.hours_per_month = hours_per_month
        self.rate = rate

    def paye_mensuelle(self):
        return self.hours_per_month*self.rate

prof1 = Professeur("Alexandra", 8, 3000, 200, 4)
prof2 = Collaborateur("David", 10, 40, 30)

# exemples
print(prof1.salaire_annuel_moyen())
print(prof2.paye_mensuelle())
```